TM1934 单线 RGB LED 驱动 · PWM+DMA 学习报告

重点：PWM+DMA “灯不亮”问题的排查与定位全过程

工程：C:\Users\Administrator\Desktop\STM32G431\TM1934 | 关键引脚：PA0→74HC245→TM1934 DIN，PF2→74HC245 DIR | 整理日期：2026-10-28

目录

1. 项目目标与硬件环境
2. 硬件连接与信号路径
3. TM1934 单线协议原理
4. PWM+DMA 驱动原理
5. **重点：PWM+DMA “灯不亮”问题排查全过程**
6. 关键时序计算
7. 函数使用详解
8. 学习方法与调试思路
9. 附录：关键代码与参数速查

1. 项目目标与硬件环境

目标：在 STM32G431VBT6 上用 **PWM + DMA** 的方式驱动 TM1934 级联 RGB 灯带（8 颗），实现无 CPU 干预的位流发送，并在此之上实现渐变、流水等动画。

项目	参数 / 说明
主控	STM32G431VBT6（LQFP100，Cortex-M4F，最高 170 MHz）
外部晶振 HSE	8 MHz
系统时钟 SYSCLK	170 MHz（PLLM=2 / PLLN=85 / PLLR=2）
定时器	TIM2（32 位通用定时器），CH1 输出 PWM
DMA	DMA1_Channel1 + DMAMUX1，由 TIM2 更新事件（Update）触发
数据引脚	PA0（TIM2_CH1，AF1）

方向引脚	PF2（74HC245 的 DIR，输出高电平）
灯带	TM1934 × 8，单线级联，RGB 恒流驱动

2. 硬件连接与信号路径

PA0 不是直接连 TM1934，而是经过一片 74HC245 缓冲器：

```
STM32 PA0 ──► 74HC245 A1 ──► B1 ──► J14.1 (LED1) ──► TM1934 DIN
STM32 PF2 ──► 74HC245 DIR
```

- `DIR = 1`：数据由 A 端流向 B 端，即 MCU → 灯板。因此 `PF2` 必须输出**高电平**，否则数据到不了 TM1934。这一步在 `TM1934_Init()` 中完成。
- 74HC245 的 `OE` 接 GND（输出始终使能）。
- 数据线空闲时应保持**低电平**；PWM 在复位段输出全 0，保证空闲低电平。
- MCU、74HC245、灯板必须共地；若 MCU 为 3.3 V 而缓冲器为 5 V，需确认缓冲器为 HCT 类（输入高阈值约 2 V），否则 3.3 V 可能无法可靠被识别。

3. TM1934 单线协议原理

3.1 位编码：高电平宽度区分 0 / 1

每个 bit 周期约 1.25 µs（800 kHz），通过**高电平持续时间**区分 0 和 1：

参数	含义	最小值	典型值	最大值
T0H	0 码高电平时间	310 ns	360 ns	410 ns
T1H	1 码高电平时间	650 ns	720 ns	1000 ns
T0/T1	单个 bit 周期	1.25 µs	1.25 µs	2.5 µs
Treset	复位低电平时间	200 µs	—	—

3.2 数据帧结构

每颗芯片 24 bit，内部顺序 **R → G → B**，每个字节 **MSB 先发**：

```
R7 R6 ... R0   G7 ... G0   B7 ... B0     ← 每颗 TM1934 共 24 bit
```

一帧 = 若干颗的 24 bit 数据 + 一个复位（低电平 > 200 μs）。复位后芯片才把收到的数据解码输出。

4. PWM+DMA 驱动原理

与 GPIO 位带（bit-bang，CPU 忙等产生精确时序）不同，本工程用定时器的 PWM 外设 + DMA 自动输出位流，CPU 只在发送前后做一次启动/停止，其余时间不参与。

4.1 基本原理

- 定时器 TIM2 配置为 PWM1 模式，周期 212 个计数（≈1.247 μs），正好是 TM1934 一个 bit 的周期。
- 每个 bit 用一个比较值 `CCR1` 表示：**0 码**写 `T0H=61`（高电平 359 ns），**1 码**写 `T1H=122`（高电平 718 ns），复位写 `0`（整周期低电平）。
- 把整帧换算成比较值数组 `pwm_buffer[]`，交给 DMA，DMA 每个定时器**更新事件**向 `CCR1` 搬一个值，实现位流的自动、连续、精确输出。
- CCR1 使能预装载（Preload）：DMA 写入的值在下一个更新事件才生效，形成“提前一拍”的流水，避免同一周期内改值导致毛刺。

4.2 为什么用更新事件而不是比较事件触发 DMA

STM32 的 `HAL_TIM_PWM_Start_DMA()` 默认用通道的**比较事件（CCxDE）**触发 DMA。当 `CCR=0`（复位码）时，比较事件与更新事件同时发生，容易丢失触发、导致整帧错位。改用**更新事件（UDE，请求号 `DMA_REQUEST_TIM2_UP`）**触发，每个周期稳定地搬一个值，这是驱动此类 LED 的推荐做法。

4.3 为什么 DMA 用 32 位（WORD）对齐

TIM2 是 **32 位定时器**，其 `CCR1` 是 32 位寄存器。DMA 的源（内存缓冲）与目的（CCR1）都必须配成 `WORD`（32 位），缓冲用 `uint32_t`。若误配 16 位（HALFWORD），对 32 位定时器会导致写入不匹配、PWM 无法正确输出。

5. 重点：PWM+DMA “灯不亮”问题排查全过程

这是本次学习最有价值的部分。现象是：**同一个硬件、同一个灯带，GPIO 位带工程（TM1934_01）正常，改成 PWM+DMA 后灯完全不亮。**下面按实际排查顺序，记录每个假设、验证与结论。

5.1 问题背景与定位前提

- 先确认硬件没问题：接线正确，参考工程能点亮，说明信号路径（PA0→74HC245→TM1934）、供电、电平都正常。
- 因此问题锁定在**软件驱动层**——从 GPIO 位带改成 PWM+DMA 时引入的若干错误。
- 排查策略：**把“协议对不对”和“新驱动写没写对”分开**，逐个假设验证，不盲改。

5.2 排查步骤与根因（按时间顺序）

① 缓冲区越界 —— 最隐蔽的“直接崩溃”

代码里复位段写 `25000` 个 0，但 `pwm_buffer` 只开了 `(N×24)+5000` 个元素。8 颗灯时实际写入 25024 个元素，越界写穿 SRAM，可能直接触发 HardFault，灯自然不亮。

根因：缓冲长度与写入长度不匹配。

修正：定义 `TM1934_RESET_COUNT 300`，缓冲大小与复位循环都用它，保证不越界。

② 时序取值贴着下限 —— 边缘性失败

原 `T0H=53` 只有约 312 ns，几乎贴着手册 310 ns 下限。经 74HC245 缓冲、走线后，高电平很容易跌破下限，导致 0 码被判错，整帧乱码。

根因：时序参数取临界值，无裕量。

修正：改用典型值 `T0H=61`（≈359 ns）、`T1H=122`（≈718 ns）。

③ DMA 触发源错误 —— 复位码触发不可靠

原用 `HAL_TIM_PWM_Start_DMA()`，它用通道**比较事件（CCxDE）**触发 DMA。当 `CCR=0`（复位码）时，比较事件与更新事件同时发生，DMA 触发不可靠，导致整帧错位。ST 官方也明确指出该方式对接近 0 或 ARR 的值不适用。

根因：触发事件与请求号不匹配。

修正：改用**更新事件（UDE）**触发，请求号 `DMA_REQUEST_TIM2_UP`，手动启动 DMA。

④ LED 数量不符 —— 数据包数量不对

参考工程 `TM1934_LED_COUNT = 8`，而 PWM+DMA 工程是 `1`。只发了 1 颗的数据，其余灯收不到数据。

根因：级联数量配置与实际颗数不符。

修正：改回 `8U`。

⑤ DMA 数据位宽错误 —— 最终修好的关键点

> TIM2 是 **32 位定时器**，其 `CCR1` 是 32 位寄存器，但 DMA 配成了 16 位（`HALFWORD`）对齐、缓冲用 `uint16_t`。位宽不匹配导致 PWM 无法正确输出。
>
> **根因**：定时器位宽与 DMA 位宽不匹配（这是 32 位定时器最容易被忽略的坑）。
>
> **修正**：改为 `WORD`（32 位）对齐，缓冲 `uint32_t`。改完编译烧录后，灯带**全亮**。

> **核心结论（一句话）**：PWM+DMA “不亮”，往往是多个“不匹配”叠加而成——**缓冲长度 vs 写入长度、时序取值 vs 手册范围、触发事件 vs 请求号、定时器位宽 vs DMA 位宽、LED 数量 vs 实际颗数**。任何一个不匹配都可能让灯完全不亮，必须逐项排除。

5.3 根因汇总表

#	现象	根因	修正
1	灯完全不亮（可能死机）	复位循环写 25000 个 0，缓冲只开 5000，越界写穿 SRAM	`TM1934_RESET_COUNT 300`，缓冲与写入长度一致
2	不亮 / 颜色乱	T0H=53（≈312 ns）贴下限 310 ns	改 61（≈359 ns，取典型值）
3	不亮 / 闪烁 / 错位	DMA 用比较事件（CCxDE）触发，CCR=0 复位时不可靠	改用更新事件 `DMA_REQUEST_TIM2_UP` + UDE
4	只亮部分 / 灯带不完整	`TM1934_LED_COUNT` 被设成 1，实际 8 颗	改为 `8U`
5	完全不亮	TIM2 是 32 位定时器，DMA 误配 16 位对齐	改为 `WORD`（32 位），缓冲 `uint32_t`

5.4 为什么这些问题“叠加”了还查了很久

- **问题不止一个**：从位带改 PWM+DMA 是一次性改了一整层，错误不是单点，而是“溢出 + 临界时序 + 触发源 + 位宽 + 数量”连环出现，修好一个还有一个。
- **硬件细节多**：32 位 vs 16 位定时器、DMAMUX 请求号、CCR 预装载、更新事件 vs 比较事件，这些不是看一句例程就能全对的。
- **“完全不亮”而非“颜色错”**：一旦最底层没输出（溢出、位宽错），表现就是全黑，掩盖了上层逻辑其实是对的。
- **静态分析有局限**：很多问题靠读代码只能“怀疑”，最终要靠示波器看 PA0 有没有 1.25 µs 波形来一锤定音。

6. 关键时序计算

系统主频 170 MHz，定时器不分频（Prescaler=0），每计数 = 1/170 MHz ≈ 5.882 ns。

项目	目标	计数 = 目标 ÷ 5.882 ns	实际取值	实际时间
T0H	360 ns	61.2	61	≈ 359 ns
T1H	720 ns	122.4	122	≈ 718 ns
周期	1.25 µs	212.5	212（ARR=211）	≈ 1247 ns
复位	> 200 µs	—	300 个 0	≈ 374 µs

7. 函数使用详解

7.1 应用层 API（`tm1934.c / tm1934.h`）

函数	作用	是否立即输出
`TM1934_Init()`	配置 PF2 方向为高、清空帧缓存并发送一帧全 0	是（发送一帧 0）
`TM1934_SetPixel(i, r, g, b)`	设置第 i 颗颜色（写帧缓存，含越界保护）	否
`TM1934_SetAll(r, g, b)`	整条灯带设为同一颜色	否
`TM1934_Clear()`	清空帧缓存（全灭）	否
`TM1934_Show()`	把帧缓存换算成 PWM 位流并经 DMA 发送	是

`TM1934_Init()`

```
void TM1934_Init(void)
{
    // 1. 使能 GPIOF 时钟，PF2 配推挽输出并拉高（74HC245 方向 A→B）
    __HAL_RCC_GPIOF_CLK_ENABLE();
    GPIO_InitStruct.Pin   = TM1934_DIR_PIN;     // PF2
    GPIO_InitStruct.Mode  = GPIO_MODE_OUTPUT_PP;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(TM1934_DIR_PORT, &GPIO_InitStruct);
    HAL_GPIO_WritePin(TM1934_DIR_PORT, TM1934_DIR_PIN, GPIO_PIN_SET);

    // 2. 上电先熄灭：清缓存 + 发一帧全 0
    TM1934_Clear();
    TM1934_Show();
}
```

`TM1934_SetPixel() / SetAll() / Clear()`

三者只操作内存帧缓存 `tm1934_buffer[N*3]`（顺序 R、G、B），不会立即改变灯的状态，必须再调 `TM1934_Show()` 才生效。

```
void TM1934_SetPixel(uint16_t index, uint8_t red, uint8_t green, uint8_t blue)
{
    if (index >= TM1934_LED_COUNT) return;      // 越界保护
    uint16_t offset = index * 3U;
    tm1934_buffer[offset]     = red;
    tm1934_buffer[offset + 1] = green;
    tm1934_buffer[offset + 2] = blue;
}
```

`TM1934_Show()`

核心发送函数：把 24 bit 数据 + 300 个复位码换算成比较值数组，然后启动 DMA 由更新事件逐个搬进 CCR1。

```
void TM1934_Show(void)
{
    uint32_t idx = 0U;

    // 1. 数据 → 比较值：1 码 = T1H，0 码 = T0H，每个字节 MSB 先发
    for (uint16_t i = 0; i < sizeof(tm1934_buffer); i++) {
        uint8_t byte = tm1934_buffer[i];
        for (int8_t bit = 7; bit >= 0; bit--)
            pwm_buffer[idx++] = (byte & (1U << bit)) ? TM1934_T1H : TM1934_T0H;
    }

    // 2. 复位码：300 个 0（约 374 µs 低电平）
    for (uint32_t r = 0; r < TM1934_RESET_COUNT; r++) pwm_buffer[idx++] = 0U;

    // 3. 停止上一次发送，复位计数器与标志
    HAL_TIM_PWM_Stop_DMA(&htim2, TIM_CHANNEL_1);
    __HAL_TIM_DISABLE_DMA(&htim2, TIM_DMA_UPDATE);
    __HAL_TIM_SET_COUNTER(&htim2, 0);
    __HAL_TIM_CLEAR_FLAG(&htim2, TIM_FLAG_UPDATE | TIM_FLAG_CC1);

    // 4. 启动 DMA（内存 → CCR1，更新事件触发），再开 UDE、CC1 输出、定时器
    HAL_DMA_Start_IT(htim2.hdma[TIM_DMA_ID_CC1],
                     (uint32_t)pwm_buffer, (uint32_t)&htim2.Instance->CCR1, idx);
    __HAL_TIM_ENABLE_DMA(&htim2, TIM_DMA_UPDATE);
    TIM_CCxChannelCmd(htim2.Instance, TIM_CHANNEL_1, TIM_CCx_ENABLE);
    __HAL_TIM_ENABLE(&htim2);
}
```

> **使用要点**：不要在中断里调用 `TM1934_Show()`；刷新频率 20–50 Hz 即可；先 `SetPixel/SetAll/Clear` 再统一 `Show`。

7.2 外设初始化函数（`tim.c / dma.c / stm32g4xx_hal_msp.c`）

函数	职责
`MX_TIM2_Init()`	初始化 TIM2 为 PWM1 模式，ARR=211（周期 212），并调用 `HAL_TIM_MspPostInit()` 配置 PA0 复用功能
`MX_DMA_Init()`	使能 DMA1/DMAMUX1 时钟、配置并打开 DMA1_Channel1 中断
`HAL_TIM_PWM_MspInit()`	使能 TIM2 时钟，初始化 DMA 句柄（请求 = TIM2_UP，方向 内存→外设，32 位对齐），并链接到 htim2
`HAL_TIM_MspPostInit()`	把 PA0 配成复用推挽 `AF1_TIM2`，高速

```
// HAL_TIM_PWM_MspInit 关键配置
hdma_tim2_ch1.Instance                 = DMA1_Channel1;
hdma_tim2_ch1.Init.Request             = DMA_REQUEST_TIM2_UP;    // 更新事件触发
hdma_tim2_ch1.Init.Direction           = DMA_MEMORY_TO_PERIPH;
hdma_tim2_ch1.Init.PeriphInc           = DMA_PINC_DISABLE;       // CCR1 固定地址
hdma_tim2_ch1.Init.MemInc              = DMA_MINC_ENABLE;        // 内存递增
hdma_tim2_ch1.Init.PeriphDataAlignment = DMA_PDATAALIGN_WORD; // 32 位
hdma_tim2_ch1.Init.MemDataAlignment    = DMA_MDATAALIGN_WORD; // 32 位
hdma_tim2_ch1.Init.Mode                = DMA_NORMAL;             // 一次性
hdma_tim2_ch1.Init.Priority            = DMA_PRIORITY_HIGH;
HAL_DMA_Init(&hdma_tim2_ch1);
__HAL_LINKDMA(htim_pwm, hdma[TIM_DMA_ID_CC1], hdma_tim2_ch1);
```

7.3 应用层动画函数（`main.c`）

函数	作用
`LED1_ColorWheel(pos, brightness, &r, &g, &b)`	把 0–255 位置映射成彩虹色并按亮度缩放
`LED1_Gradient(tick)`	整条灯带空间渐变 + 时间亮度呼吸（三角波）
`LED1_Running(tick)`	带拖尾的流水灯；单颗时退化为颜色循环

```
// 主循环：每 8 秒切换一种动画
uint32_t tick = HAL_GetTick();
if (((tick / LED1_EFFECT_PERIOD_MS) & 1U) == 0U) LED1_Gradient(tick);
else                                             LED1_Running(tick);
HAL_Delay(LED1_FRAME_MS);   // 20 ms 一帧
```

8. 学习方法与调试思路

1. **先有可运行参考：**从一个可工作的 GPIO 位带工程（`TM1934_01`）入手，确认硬件连线、供电、时序参数正确，把“协议是否正确”与“新驱动是否写对”两个问题分开。
2. **读通协议：**对照 TM1934 数据手册，明确位编码（T0H/T1H）、数据顺序（RGB、MSB 先发）、复位时间（>200 µs）三大要素。
3. **理解外设：**搞清 PWM 模式如何用 `CCR` 控制占空比，DMA 如何“每个周期搬一个比较值”，以及预装载带来的“延迟一拍”流水。
4. **逐步替换：**只把发送层从“GPIO 位带”替换成“PWM+DMA”，应用层动画函数保持不变，便于对比排查。
5. **参数要留裕量：**时序取数据手册典型值，而不是贴着最小/最大值；否则经过缓冲器、走线后容易超差。
6. **用示波器/逻辑分析仪验证：**先看 PA0 有没有 1.25 µs 周期的波形，再判断是“没输出”还是“输出但灯不响应”。

9. 附录：关键代码与参数速查

9.1 参数定义（`tm1934.c` 顶部）

```
#define TM1934_DIR_PORT    GPIOF
#define TM1934_DIR_PIN     GPIO_PIN_2

#define TM1934_T0H         61U     // 0 码高电平 ≈ 359 ns
#define TM1934_T1H         122U    // 1 码高电平 ≈ 718 ns
#define TM1934_PERIOD      212U    // 位周期（ARR=211）
#define TM1934_RESET_COUNT 300U    // 复位 0 的个数 ≈ 374 µs
```

9.2 级联数量（`tm1934.h`）

```
#ifndef TM1934_LED_COUNT
#define TM1934_LED_COUNT 8U
#endif
```

9.3 RAM 与刷新时间估算

级联数	帧缓存	pwm_buffer（uint32_t）	单帧数据时间
8	24 B	(8×24+300)×4 = 1968 B	192×1.247 µs ≈ 240 µs
256	768 B	(256×24+300)×4 ≈ 25.8 KB	6144×1.247 µs ≈ 7.66 ms

> **提示：**级联数量很大时，pwm_buffer 会占较多 RAM，且每帧发送时间变长。若需驱动上百颗，可进

一步优化（如只存位流、缩短复位、或用 SPI 外设映射）。